

Guide to Building a Backend System with Nestjs

1. Giới thiệu về NestJS

NestJS là gì?

- NestJS là một framework Node.js progressive, mã nguồn mở, được xây dựng trên TypeScript, lấy cảm hứng từ kiến trúc Angular. Framework sử dụng kết hợp OOP (Object Oriented Programming), FP (Functional Programming) và FRP (Functional Reactive Programming), chạy trên Express hoặc Fastify.
- NestJS phù hợp với việc xây dựng các ứng dụng backend quy mô enterprise: REST API, GraphQL API, Microservices, WebSocket, và event-driven systems. Hiện có hơn 75.000 GitHub stars và hơn 3 triệu weekly npm downloads.

Tại sao chọn NestJS?

- **Kiến trúc rõ ràng:** Module, Controller, Service, Provider – cấu trúc theo chuẩn enterprise, dễ mở rộng và bảo trì.
- **TypeScript-first:** Type safety từ API request đến database, giảm thiểu runtime errors.
- **Dependency Injection:** DI container tích hợp, dễ test và loose coupling giữa các component.
- **Decorator-driven:** @Controller, @Get, @Injectable, @Module – khai báo rõ ràng, quen thuộc với dev Angular/Java Spring.
- **Ecosystem phong phú:** Tích hợp sẵn Prisma, TypeORM, GraphQL, gRPC, WebSocket, Kafka, Bull, Passport, Swagger.
- **Performance:** NestJS 11 với SWC compiler: build nhanh hơn 20x; trên Fastify adapter đạt ~12.000 QPS.
- **Enterprise-proven:** Sử dụng bởi Roche, Capgemini, Adidas, và hàng nghìn công ty trên thế giới.

2. Goals After 1 Month

Basic Skills

- Cài đặt và cấu hình môi trường phát triển NestJS 11 + Node.js 22 LTS.
- Hiểu kiến trúc NestJS: Module, Controller, Service, Provider, Middleware, Guard, Interceptor, Pipe.
- Xây dựng REST API hoàn chỉnh với validation, error handling chuẩn.
- Làm việc với database dùng Prisma (recommended) hoặc TypeORM.
- Implement authentication với JWT, Passport.js, và refresh token.
- Sử dụng Guard và Role-based access control.
- Viết unit test và e2e test với Vitest (hoặc Jest).
- Containerize và deploy ứng dụng NestJS với Docker.
- Xây dựng real-time features với WebSockets (@nestjs/websockets).
- Caching với Redis và queue với BullMQ.

Practical Deliverables

- CRUD API hoàn chỉnh với NestJS + Prisma + PostgreSQL.
- Hệ thống authentication: JWT access token + refresh token + OAuth2.
- API có phân quyền RBAC (admin, user, moderator).
- Ứng dụng real-time chat với WebSocket và Redis Pub/Sub.
- Background job processing với BullMQ.
- Deploy production-ready với Docker + CI/CD pipeline.

3. Learning Content

Basic Knowledge

- TypeScript nâng cao: decorators, generics, utility types, strict mode.
- Node.js internals: event loop, async/await, streams, modules (ESM vs CJS).
- NestJS core concepts: DI container, lifecycle hooks, module system.
- HTTP fundamentals: REST conventions, status codes, headers, CORS.
- SQL cơ bản và nâng cao: JOIN, transaction, indexing, query optimization.
- Unit test và integration test theo TDD approach.
- API security: OWASP Top 10, input sanitization, rate limiting, HTTPS.

Tools and Libraries

Công cụ / Thư viện	Mục đích	Ghi chú
Node.js 22 LTS	Runtime	Active LTS cho đến tháng 4/2027; hỗ trợ native ESM, Web Streams
NestJS 11.x	Framework chính	Stable 2026; v12 (ESM, Vitest, oxlint) dự kiến Q3 2026
TypeScript 5.x	Ngôn ngữ	Strict mode bắt buộc cho enterprise projects
Prisma 7	ORM (recommended)	Schema-first, pure TypeScript client (không còn Rust engine từ v7)
Drizzle ORM	ORM (alternative)	SQL-first, ~7.4kb, zero deps, tốt cho serverless/edge
TypeORM	ORM (legacy/existing)	Dùng nếu codebase đã có; không khuyến khích cho project mới
PostgreSQL 16+	RDBMS	Database chính, kết hợp tốt với Prisma/Drizzle
Redis 7+	Cache & Pub/Sub	Session, caching, queue, real-time messaging
BullMQ	Job Queue	Background jobs, scheduled tasks; thay thế Bull (deprecated)
Passport.js	Authentication	Local, JWT, OAuth2 strategies; tích hợp native với NestJS
class-validator / Zod	Validation	class-validator: NestJS default; Zod: NestJS v12 Standard Schema
Swagger (OpenAPI)	API Documentation	@nestjs/swagger tự động generate docs từ decorators
Socket.io / WS	WebSocket	@nestjs/websockets adapter; Socket.io cho rooms & namespaces
gRPC	Microservice RPC	@nestjs/microservices; high-perf cho inter-service communication
Kafka / NATS	Message broker	Kafka: enterprise; NATS: cloud-native lightweight

Công cụ / Thư viện	Mục đích	Ghi chú
Docker + Kubernetes	Container & Orchestration	Multi-stage build, K8s deployment cho production

Workflow

- Thiết kế module theo feature (User, Auth, Product, Order) thay vì theo layer.
- Dùng Git + GitHub/GitLab với conventional commits và branch strategy (GitFlow hoặc trunk-based).
- API versioning (/api/v1/, /api/v2/) và backward compatibility.
- Docker multi-stage build để tối ưu image size cho production.
- CI/CD với GitHub Actions: lint, test, build, docker push, deploy.
- Monitoring với Prometheus + Grafana + nestjs-prometheus.
- Centralized logging với Pino/Winston + ELK Stack hoặc Loki.
- Health check endpoint với @nestjs/terminus.
- Secrets management: không hardcode, dùng .env + @nestjs/config + Vault/cloud secrets.

4. Weekly Learning Plan

4.1 Training Approach

E-commerce Order Management Backend

Đây là hệ thống backend mô phỏng nghiệp vụ quản lý sản phẩm, khách hàng, giỏ hàng, đơn hàng, thanh toán giả lập và thông báo trạng thái đơn hàng. Domain này phù hợp với NestJS vì có nhiều module độc lập, nhiều business rule, nhiều guard/interceptor/pipe, và dễ mở rộng sang microservices hoặc event-driven architecture.

Các module chính trong project gồm:

Module	Mục đích
Auth	Đăng ký, đăng nhập, JWT, refresh token

User	Quản lý tài khoản và thông tin người dùng
Role & Permission	Phân quyền admin, staff, customer
Product	Quản lý sản phẩm
Category	Quản lý danh mục sản phẩm
Inventory	Quản lý tồn kho
Cart	Quản lý giỏ hàng
Order	Tạo và xử lý đơn hàng
Payment	Mô phỏng thanh toán
Notification	Gửi thông báo qua background job
Realtime	Cập nhật trạng thái đơn hàng realtime
Audit Log	Ghi nhận hành động quan trọng trong hệ thống

Mục tiêu cuối cùng là sinh viên hiểu cách một **NestJS** backend được tổ chức theo module, cách các component tương tác qua Dependency Injection, cách áp dụng Guard, Pipe, Interceptor, Prisma, Redis, BullMQ, Swagger, Docker và testing trong một project thực tế.

4.2 Expected Outcome After 1 Month

Sau 1 tháng, sinh viên cần đạt được các năng lực sau:

Nhóm năng lực	Kết quả mong đợi
TypeScript Foundation	Hiểu strict mode, type, interface, class, decorator, generic cơ bản
NestJS Architecture	Hiểu Module, Controller, Service, Provider, Guard, Pipe, Interceptor
REST API	Xây dựng API chuẩn REST, versioning, response format rõ ràng
Validation	Dùng DTO, class-validator hoặc schema validation
Database	Làm việc với PostgreSQL thông qua Prisma
Authentication	Implement JWT access token, refresh token, protected route
Authorization	Áp dụng Role-based Access Control cho admin/staff/customer

API Documentation	Generate Swagger/OpenAPI document
Caching	Dùng Redis cache cho product/category/order query
Queue	Dùng BullMQ để xử lý notification hoặc order processing async
Realtime	Dùng WebSocket Gateway để cập nhật trạng thái đơn hàng
Testing	Viết unit test và e2e test cơ bản
DevOps	Dockerize ứng dụng và chạy bằng Docker Compose
Workflow	Làm việc theo branch, commit convention, pull request và code review

4.3 Weekly Plan Overview

Week	Focus	Main Output
Week 1	NestJS foundation, module design, REST API	Product & Category API

Week 2	Prisma, Auth, RBAC, Swagger	User/Auth/RBAC + protected Product Admin API
Week 3	Cart, Order, Inventory, BullMQ, Redis	Order flow có cache và background job
Week 4	WebSocket, testing, Docker, CI/CD, final demo	E-commerce backend hoàn chỉnh

4.4 Week 1 – Go Fundamentals & REST API Foundation

Objective

Tuần đầu tiên tập trung vào việc giúp hiểu cấu trúc cốt lõi của NestJS: module, controller, service, provider, DTO, validation, dependency injection và REST API convention.

Ở tuần này chưa cần authentication và database phức tạp. Có thể dùng in-memory data hoặc database đơn giản tùy năng lực. Trọng tâm là hiểu cách NestJS tổ chức code theo feature module.

Innovate The Future

Key Topics

Topic	Nội dung cần hiểu
NestJS CLI	Tạo project, generate module/controller/service
Module system	Tổ chức code theo feature module

Controller	Nhận request, map route, trả response
Service	Chứa business logic
Provider & DI	Inject dependency qua constructor
DTO	Định nghĩa input contract
Validation Pipe	Validate request body/query/params
Exception Filter	Chuẩn hóa lỗi trả về
Interceptor	Chuẩn hóa response hoặc logging
API versioning	Thiết kế route theo /api/v1
Swagger basic	Generate API docs từ decorators

Practical Tasks

Task ID	Task	Description	Expected Output
---------	------	-------------	-----------------

N1-T01	Setup NestJS project	Tạo project NestJS với TypeScript strict mode	Project chạy được local
N1-T02	Setup project structure	Tổ chức folder theo feature module	Codebase rõ ràng ngay từ đầu
N1-T03	Create Product module	Tạo module, controller, service cho Product	Có Product API cơ bản
N1-T04	Create Category module	Tạo module, controller, service cho Category	Có Category API cơ bản
N1-T05	Add DTO validation	Validate create/update product và category	Request sai bị reject rõ ràng
N1-T06	Add global response interceptor	Chuẩn hóa response format toàn hệ thống	Response thống nhất
N1-T07	Add global exception filter	Chuẩn hóa error format	Error response dễ đọc
N1-T08	Add Swagger documentation	Tích hợp Swagger cho Product/Category API	API docs chạy được
N1-T09	Add basic unit tests	Test service logic cho Product/Category	Có test cơ bản

N1-T10	Mini assessment	Sinh viên tự thêm Brand module theo pattern đã học	Kiểm tra khả năng áp dụng lại module pattern
--------	-----------------	--	--

Week 1 Deliverables

Deliverable	Description
NestJS source code	Project có cấu trúc module rõ ràng
Product API	CRUD product cơ bản
Category API	CRUD category cơ bản
DTO validation	Validate request body/query
Global response format	Response thống nhất
Swagger docs	Có API documentation
Unit tests	Test service logic cơ bản
README	Hướng dẫn chạy project local

4.5 Week 2 – Database, Authentication & Clean Architecture

Objective

Tuần 2 chuyển project sang database thật với Prisma và PostgreSQL. Học cách thiết kế **schema, migration, seed data, repository/service pattern** trong NestJS, đồng thời **implement authentication và phân quyền**.

Tuần 2 bắt đầu áp dụng security và enterprise backend pattern: **JWT, refresh token, Guard, custom decorator, RBAC, Prisma transaction và Swagger** cho protected API.

Key Topics

Topic	Nội dung cần hiểu
Prisma schema	Thiết kế model, relation, enum
Prisma migration	Quản lý thay đổi database
Prisma Client	Query database type-safe
Seed data	Tạo dữ liệu mẫu cho development
Auth module	Register, login, refresh token
Password hashing	Hash password trước khi lưu
Passport JWT strategy	Xác thực request bằng JWT

Guard	Bảo vệ route
Custom decorator	Lấy current user từ request
RBAC	Phân quyền admin, staff, customer
Prisma transaction	Xử lý nhiều thao tác DB trong một transaction

Practical Tasks

Task ID	Task	Description	Expected Output
N2-T01	Setup PostgreSQL	Chạy PostgreSQL bằng Docker Compose	Database local hoạt động
N2-T02	Setup Prisma	Cấu hình Prisma, datasource, generator	Prisma Client hoạt động
N2-T03	Design Prisma schema	Tạo model User, Role, Product, Category, Inventory	Schema có relation rõ
N2-T04	Add migration	Tạo migration đầu tiên	Có thể recreate DB

N2-T05	Add seed data	Seed admin user, category, product mẫu	Có dữ liệu test nhanh
N2-T06	Build Auth module	Register, login, refresh token	Auth flow hoạt động
N2-T07	Add JWT Guard	Bảo vệ route cần đăng nhập	Unauthorized request bị chặn
N2-T08	Add Roles Guard	Phân quyền theo role	Admin/staff/customer có quyền khác nhau
N2-T09	Protect Product Admin API	Chỉ admin/staff được tạo/sửa/xóa product	Customer chỉ xem được product
N2-T10	Add audit log basic	Ghi log khi admin tạo/sửa/xóa product	Có audit trail cơ bản
N2-T11	Add Swagger auth config	Swagger hỗ trợ Bearer token	Test protected API từ Swagger được

Suggested Database Entities

Entity	Purpose	Main Fields
--------	---------	-------------

User	Tài khoản người dùng	id, email, passwordHash, fullName, role, status
Product	Sản phẩm	id, name, sku, price, status, categoryId
Category	Danh mục	id, name, slug, isActive
Inventory	Tồn kho	id, productId, quantity, reservedQuantity
AuditLog	Lịch sử thao tác	id, actorId, action, entityType, entityId, metadata

Suggested RBAC Rules

Action	Admin	Staff	Customer
View product	Yes	Yes	Yes
Create product	Yes	Yes	No
Update product	Yes	Yes	No
Delete product	Yes	No	No

View audit log	Yes	No	No
Manage users	Yes	No	No

Week 2 Deliverables

Deliverable	Description
Prisma schema	Có model và relation rõ ràng
Migration files	Database có thể setup lại từ đầu
Seed script	Có dữ liệu mẫu
Auth API	Register, login, refresh token
JWT Guard	Protected route hoạt động
Roles Guard	Phân quyền admin/staff/customer
Admin Product API	Quản lý product theo role

Swagger protected docs	Swagger dùng được với Bearer token
------------------------	------------------------------------

4.6 Week 3 – Redis, Background Jobs & Realtime

Objective

Tuần 3 tập trung vào business flow thực tế của ecommerce: thêm sản phẩm vào giỏ hàng, tạo đơn hàng, kiểm tra tồn kho, giữ hàng, cập nhật trạng thái đơn và xử lý notification bằng background job.

Key Topics

Topic	Nội dung cần hiểu
Cart flow	Add item, update quantity, remove item
Order lifecycle	Pending, paid, processing, shipped, completed, cancelled
Inventory reservation	Kiểm tra và giữ tồn kho khi tạo order
Transaction	Tạo order và order items atomic
Redis cache	Cache product listing và category tree
Cache invalidation	Xóa cache khi product/category thay đổi
BullMQ	Queue xử lý notification hoặc order timeout

Scheduled job	Hủy đơn chưa thanh toán sau thời gian cấu hình
Idempotency	Tránh tạo trùng order hoặc payment
Domain event	Tách event order.created, order.paid, order.cancelled

Practical Tasks

Task ID	Task	Description	Expected Output
N3-T01	Build Cart module	Customer thêm/xóa/cập nhật sản phẩm trong giỏ	Cart API hoạt động
N3-T02	Build Order module	Customer tạo order từ cart	Order được tạo từ cart
N3-T03	Add OrderItem model	Lưu snapshot product price/name tại thời điểm mua	Order không bị sai khi product đổi giá
N3-T04	Add inventory check	Không cho order vượt tồn kho	Business rule tồn kho hoạt động
N3-T05	Add inventory reservation	Khi tạo order thì reserved quantity tăng	Tồn kho được giữ tạm

N3-T06	Add order status workflow	Quản lý trạng thái đơn hàng	Không chuyển trạng thái sai
N3-T07	Add fake payment module	Mô phỏng thanh toán thành công/thất bại	Order chuyển paid/cancelled
N3-T08	Add Redis product cache	Cache danh sách product/category	API list product nhanh hơn
N3-T09	Add cache invalidation	Khi admin update product/category thì xóa cache	Cache không stale
N3-T10	Add BullMQ notification queue	Khi order created/paid/cancelled thì enqueue notification	Job xử lý async
N3-T11	Add order timeout job	Đơn pending quá hạn thì tự cancel	Scheduled/delayed job hoạt động
N3-T12	Add idempotency key	Tránh submit payment/order nhiều lần	Request lặp không tạo dữ liệu trùng

Suggested Order Status

Status	Meaning
--------	---------

pending	Đơn mới tạo, chưa thanh toán
paid	Đã thanh toán
processing	Đang xử lý
shipped	Đã giao cho đơn vị vận chuyển
completed	Hoàn tất
cancelled	Đã hủy
refunded	Đã hoàn tiền

Suggested Business Rules

Rule ID	Rule
BR-01	Customer chỉ được tạo order từ cart của chính mình
BR-02	Không cho tạo order nếu sản phẩm không active

BR-03	Không cho tạo order nếu số lượng vượt tồn kho khả dụng
BR-04	Khi order được tạo, hệ thống phải reserve inventory
BR-05	Khi payment thành công, order chuyển từ pending sang paid
BR-06	Khi order bị cancel, reserved inventory phải được release
BR-07	Không cho chuyển trạng thái từ completed về pending
BR-08	Payment request phải có idempotency key
BR-09	Order pending quá thời gian cấu hình sẽ tự động cancelled
BR-10	Khi order status thay đổi, hệ thống phải ghi audit log

Innovate The Future

Suggested Events

Event	Trigger	Consumer
order.created	Customer tạo order	Notification queue, audit log

order.paid	Payment thành công	Notification queue, inventory service
order.cancelled	Order bị hủy	Inventory release, notification queue
product.updated	Admin cập nhật product	Cache invalidation
inventory.low_stock	Tồn kho thấp	Notification queue

Week 3 Deliverables

Deliverable	Description
Cart API	Add/update/remove cart item
Order API	Tạo order từ cart
Inventory logic	Check và reserve tồn kho
Payment simulation	Thanh toán giả lập
Redis cache	Cache product/category

BullMQ worker	Xử lý notification async
Order timeout job	Tự hủy order pending quá hạn
Idempotency handling	Tránh tạo payment/order trùng

4.7 Week 4 – Testing, Docker, CI/CD & Final Demo

Objective

Tuần 4 hoàn thiện hệ thống theo chuẩn gần production. Bổ sung WebSocket realtime update, unit test, e2e test, health check, logging, Docker, CI pipeline và final demo.

Key Topics

Topic	Nội dung cần hiểu
WebSocket Gateway	Realtime update bằng @nestjs/websockets
Socket rooms	Subscribe theo user hoặc order
E2E testing	Test API flow end-to-end
Unit testing	Test service bằng mocked dependencies

Health check	Kiểm tra database, Redis, service status
Structured logging	Log request, error, job status
Metrics basic	Request count, latency, queue jobs
Docker multi-stage	Build production image
Docker Compose	Chạy API, worker, PostgreSQL, Redis
CI pipeline	Lint, test, build tự động

Practical Tasks

Task ID	Task	Description	Expected Output
N4-T01	Add WebSocket Gateway	Client connect vào order realtime gateway	WebSocket hoạt động
N4-T02	Add order room	Customer subscribe theo orderId hoặc userId	Chỉ nhận event liên quan

N4-T03	Broadcast order status	Khi order status đổi, client nhận event realtime	Realtime order tracking
N4-T04	Add unit tests	Test Auth, Product, Cart, Order service	Service logic có test
N4-T05	Add e2e tests	Test flow register, login, add cart, create order, pay order	Main flow được verify
N4-T06	Add permission tests	Test customer không được dùng admin API	RBAC được kiểm tra
N4-T07	Add health check	Kiểm tra app, database, Redis	Có endpoint health
N4-T08	Add structured logging	Log request, error, order event, job event	Log dễ trace
N4-T09	Add Dockerfile	Build NestJS app bằng multi-stage Dockerfile	Image build được
N4-T10	Complete Docker Compose	Chạy API, worker, PostgreSQL, Redis	Full system chạy bằng một command

N4-T11	Add CI pipeline	Lint, test, build tự động	Pull request được kiểm tra
N4-T12	Prepare final demo	Chuẩn bị demo full ecommerce flow	Demo hoàn chỉnh

Final Demo Flow

Step	Action	Expected Result
1	Register customer	Tạo tài khoản customer
2	Login customer	Nhận access token và refresh token
3	Admin login	Đăng nhập tài khoản admin seed sẵn
4	Admin create product	Product được tạo và inventory được set
5	Customer view products	Product listing dùng Redis cache
6	Customer add to cart	Cart được cập nhật
7	Customer create order	Order được tạo, inventory được reserve

8	Customer pay order	Payment giả lập thành công
9	Order status updated	Order chuyển sang paid
10	Notification job processed	BullMQ worker xử lý notification
11	WebSocket event received	Client nhận trạng thái order realtime
12	Run tests	Unit/e2e test pass
13	Run Docker Compose	Full system chạy local

Week 4 Deliverables

Deliverable	Description
WebSocket Gateway	Realtime order tracking
Unit tests	Test service logic
E2E tests	Test main API flow

Health endpoint	Kiểm tra trạng thái service
Structured logs	Log request/job/event
Dockerfile	Build production image
docker-compose.yml	Chạy full system
CI pipeline	Test/lint/build tự động
README.md	Hướng dẫn setup, run, test
Final demo	Demo hệ thống hoàn chỉnh

4.8 Optional Advanced Tasks

Dành cho sinh viên làm nhanh hoặc có năng lực tốt hơn.

Level	Task	Description
Level 1	Product Search & Filtering	Thêm filter theo category, price range, keyword, status

Level 2	Pagination Standardization	Chuẩn hóa pagination response cho toàn bộ API
Level 3	Coupon Module	Thêm mã giảm giá theo rule fixed/percentage/min order value
Level 4	Address Module	Customer quản lý nhiều địa chỉ giao hàng
Level 5	Order Refund Flow	Thêm refund giá lập và trạng thái refunded
Level 6	Audit Log Advanced	Ghi log mọi hành động admin quan trọng
Level 7	Soft Delete	Product/category dùng soft delete thay vì hard delete
Level 8	Outbox Pattern	Lưu domain event vào DB trước khi publish queue
Level 9	Multi-tenant Store	Hỗ trợ nhiều store/vendor
Level 10	GraphQL API	Thêm GraphQL endpoint cho product catalog
Level 11	NATS/Kafka Event Bus	Publish order event qua message broker

Level 12	Prometheus Metrics	Thêm metrics cho request latency, order count, queue jobs
----------	--------------------	---



TEKNIX

Innovate The Future